

**Name: Shaikh Mohammed Saad**

**Roll no: B-39**

**Branch:- AI&DS(B)**

**Experiment:- 04**

**AIM:-Data Wrangling with Pandas: Manipulate and transform data to suit the analysis need.**

**Q1. Explain the role of NumPy in the Python scientific computing ecosystem.**

**ANS:-** NumPy (Numerical Python) is the core library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on them. Its key roles include:

- Acting as the foundation for many other scientific libraries.
- Enabling efficient storage and manipulation of numerical data
- Supporting vectorized operations, reducing the need for loops.
- Serving as the base for libraries like Pandas, Matplotlib, and Scikit-learn.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print("NumPy Array:", arr)
print("Array Type:", type(arr))
```

```
NumPy Array: [1 2 3 4 5]
Array Type: <class 'numpy.ndarray'>
```

**Q2. Why is NumPy considered faster than pure Python computations?**

**ANS:-** NumPy is faster than pure Python because:

- Vectorization: Operations are applied to entire arrays at once instead of element-by-element loops
- Optimized C Implementation: NumPy functions are written in C, making them highly efficient.
- Less Overhead: Python lists store mixed data types, while NumPy arrays store homogeneous data, reducing memory and computation overhead
- Better Memory Usage: Data is stored in contiguous memory blocks, improving access speed

```
import numpy as np
import time
lst = list(range(1000000))
start = time.time()
lst = [x * 2 for x in lst]
print("Python Time:", time.time() - start)
arr = np.arange(1000000)
start = time.time()
arr = arr * 2
print("NumPy Time:", time.time() - start)
```

```
Python Time: 0.12811851501464844
NumPy Time: 0.005824089050292969
```

### Q3.What types of mathematical operations can be performed using NumPy?

**ANS:-** NumPy supports a wide range of operations:

- Arithmetic operations: +, -, ×, ÷
- Statistical operations: mean, median, standard deviation.
- Linear algebra: dot product, matrix multiplication.
- Trigonometric functions: sin, cos, tan.
- Logarithmic & exponential functions

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print("Addition:", arr + 2)
print("Mean:", np.mean(arr))
print("Sine:", np.sin(arr))
print("Exponential:", np.exp(arr))
```

```
Addition: [3 4 5 6]
Mean: 2.5
Sine: [ 0.84147098  0.90929743  0.14112001 -0.7568025 ]
Exponential: [ 2.71828183  7.3890561  20.08553692  54.59815003]
```

### Q4. Explain indexing and slicing in NumPy arrays.

**ANS:-** Indexing and slicing allow accessing specific elements or subsets of arrays.

- Indexing: Access single elements
- Slicing: Extract portions of arrays

Example:-

- `arr[0]` → first element
- `arr[1:4]` → elements from index 1 to 3

NumPy also supports:

- Multi-dimensional indexing

- ♦ Boolean indexing

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("First Element:", arr[0])
print("Elements from index 1 to 3:", arr[1:4])
```

```
First Element: 10
Elements from index 1 to 3: [20 30 40]
```

**Q5. How does NumPy integrate with libraries like:**

**ANS:-** NumPy integrates seamlessly with other Python libraries, making it a core component of the data science ecosystem.

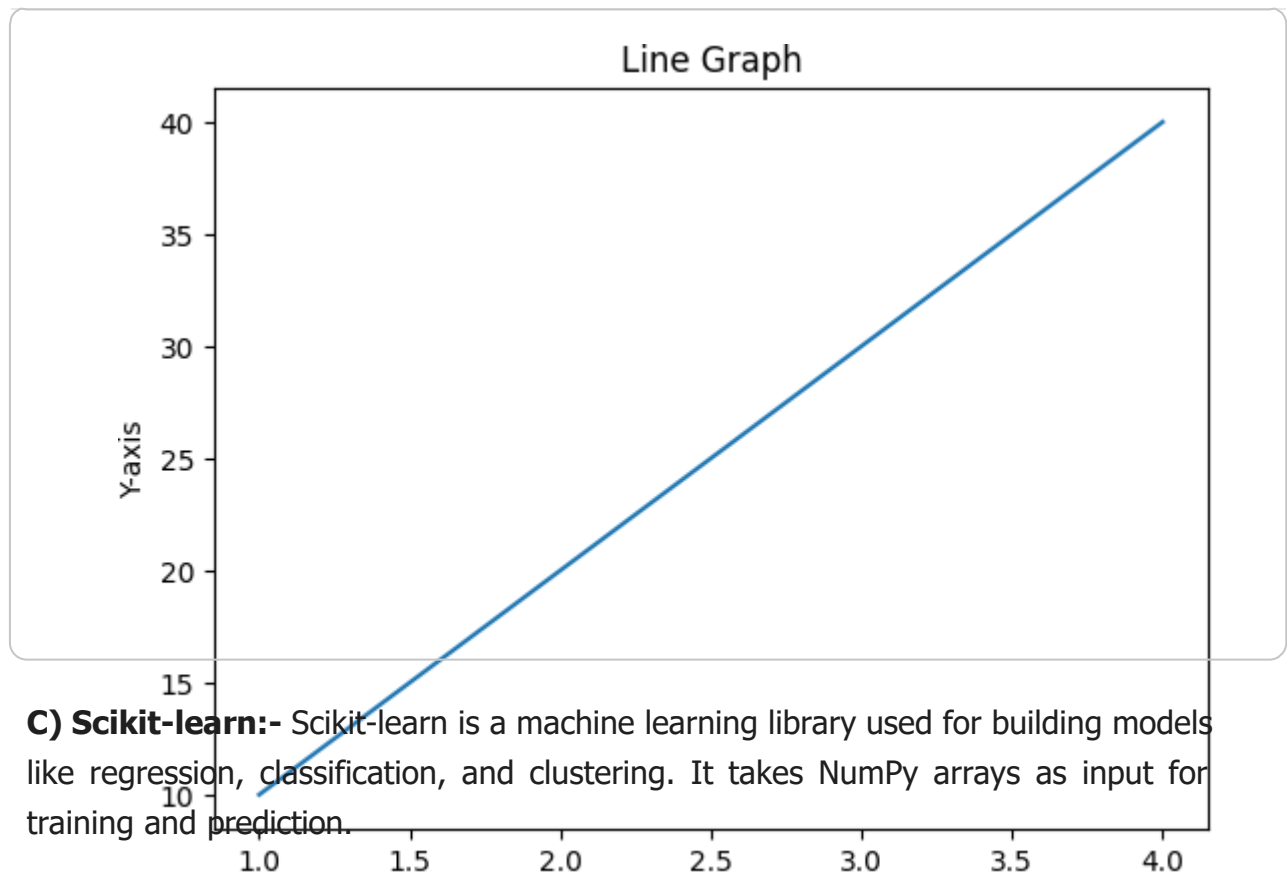
**A) Pandas:-** Pandas is a data analysis library that provides data structures like DataFrames and Series. It uses NumPy arrays internally for fast data processing and manipulation.

```
import numpy as np
import pandas as pd
data = np.array([10, 20, 30, 40])
df = pd.DataFrame(data, columns=["Values"])
print(df)
```

|   | Values |
|---|--------|
| 0 | 10     |
| 1 | 20     |
| 2 | 30     |
| 3 | 40     |

**B) Matplotlib:-** Matplotlib is a data visualization library used to create graphs, charts, and plots. It uses NumPy arrays to represent numerical data for plotting.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4])
y = np.array([10, 20, 30, 40])
plt.plot(x, y)
plt.title("Line Graph")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1], [2], [3], [4]])
y = np.array([10, 20, 30, 40])
model = LinearRegression()
model.fit(X, y)
print("Prediction for 5:", model.predict([[5]]))
```

Prediction for 5: [50.]